| | | |
|---|---|---|
| | **S3.01 Conception et Développement d'application**<br>**PARTIE JAVA-PYTHON**<br>Enseignants référents pour cette partie : Patricia Stolf (Python), André Péninou (Java) | |

# Visualisation de résultats d'ordonnancement de processus

## I. Présentation

L'ensemble des processus d'une machine sont ordonnancés par le système d'exploitation. Le rôle de l'ordonnanceur est de choisir quand (à quelle date et pour combien de temps) et sur quel processeur s'exécutent les différents processus.
L'objectif de la SAé est de visualiser et de comparer plusieurs algorithmes d'ordonnancement sur un même ensemble de processus.

## II. Vos missions

**Une application Python** devra implémenter plusieurs algorithmes d'ordonnancement et écrire les résultats de l'ordonnancement dans des fichiers. Une application Java devra permettre de paramétrer le comportement du programme Python (via un fichier de configuration lu par ce dernier) et récupérera les résultats de l'ordonnancement via les fichiers pour affichage.

**Utilisateurs visés** L'objectif des utilisateurs utilisant l'application à développer est de visualiser et de comparer plusieurs algorithmes d'ordonnancement sur un même ensemble de processus.
L'utilisateur a des notions de processus, processeur, RAM, etc. utilisées dans l'application. Il cherche à comprendre les politiques d'ordonnancement, en particulier en comparant des résultats (globaux, détaillés, etc.). Des visualisations simultanées pour plusieurs algorithmes sont attendues pour pouvoir effectuer ces comparaisons.
On peut imaginer un étudiant de lycée ou de 2° année qui cherche à comprendre le fonctionnement des processus après un exposé introductif (cours amphi, etc.).

**Le programme Java** proposera plusieurs interfaces :

1) L'interface doit donner la possibilité de modifier le fichier de configuration de l'ordonnanceur :
    - choisir le fichier contenant la liste des processus à ordonnancer,
    - choisir le fichier contenant la description des ressources disponibles pour exécuter les processus (nombre de processeurs, taille de la RAM)
    - choisir le/les fichiers dans lequel les résultats de l'ordonnancement seront stockés (on conseille de séparer par algorithme d'ordonnancement)
    - choisir le fichier dans lequel les métriques globales seront stockées
    - choisir le ou les algorithmes d'ordonnancement que le programme python exécutera pour ordonnancer les processus. Si l'algorithme d'ordonnancement a besoin d'un quantum de temps, la valeur est aussi dans le fichier de configuration.
2) Le programme Java doit lancer le programme python
3) L'interface devra permettre plusieurs affichages :
    - affichage de la liste des processus avec les informations décrivant les processus (décrit plus bas dans la partie III).
    - affichage pour chaque processus de sa date de soumission, date de début d'exécution et date de fin d'exécution
    - affichage d'un graphe au cours du temps des processus (un ou plusieurs de façon paramétrable) montrant les dates de début, de fin, les temps d'exécution et le processeur utilisé, ainsi que les moments où ils sont en attente
    - affichage d'un graphe au cours du temps pour chaque processeur (ou plusieurs et de façon paramétrable), pour une fenêtre de temps donnée, des processus s'exécutant sur ce processeur.
    - affichage des métriques générales par algorithme et un graphique qui permet de les comparer : temps de réponse moyen, temps d'attente moyen, date de fin du dernier processus à s'exécuter (appelé makespan), ... (cf. métriques partie Python)
    - affichage des métriques d'énergie consommée/énergie disponible par pas de temps, soit sur un graphique séparé, soit dans les graphiques précédents (mieux).

**Quant à lui, le programme Python sert à :**

1) lire le fichier de configuration pour savoir quel comportement adopter.

2) exécuter le ou les algorithmes d'ordonnancement. On attend au minimum deux politiques implémentées. Des exemples sont donnés plus bas, vous pouvez en imaginer d'autres.

Les politiques sont contraintes par :

- la RAM : la RAM totale utilisée par les processus en exécution ne devra pas dépasser la RAM total disponible
- la disponibilité d'énergie renouvelable disponible. Dans un centre de calcul « vert », on cherche à faire tourner les processus quand l'énergie renouvelable est disponible. Cette énergie n'est pas constante : elle suit un profil (beaucoup de solaire à midi, peu la nuit). Quand l'énergie est faible, on ne peut alimenter qu'une partie des processeurs ; quand elle est abondante, on peut tous les utiliser. Le profil d'énergie renouvelable est fourni dans un fichier d'entrée décrit plus bas. On considère qu'il est impossible de consommer plus que l'énergie verte disponible.

3) écrire les résultats de l'ordonnancement et les métriques dans les fichiers.

L'écriture des fichiers est gérée par le programme Python. Le programme Java se focalise sur l'affichage (la visualisation).

Pour la démonstration lors de l'oral, c'est le programme Java qui lancera le programme python. Cependant, il vous est demandé de déposer un programme Python exécutable seul en ligne de commande.

**Structure du livrable du code Python imposée : vous devez livrer le code python en respectant impérativement l'arborescence des fichiers ci-dessous :**

- Code : vous structurez ce répertoire comme souhaité, il doit contenir, les algos, les codes de lecture/écriture des fichiers
- Inputs : ce répertoire doit contenir des sous-répertoires séparés par test et pour chaque test, on doit trouver tous les fichiers d'entrée
    - Test1 : fichiers ressources.json, config.txt, processus.csv, energie.csv
    - Test2 : fichiers ressources.json, config.txt, processus.csv, energie.csv
    - ...

– Outputs : ce répertoire doit contenir des sous-répertoires séparés par test et pour chaque test, on doit trouver tous les fichiers de sortie
    - Test1 : fichier allocation globale par politique, fichier allocation détaillée par politique, fichier métriques
    - Test2 : fichier allocation globale par politique, fichier allocation détaillée par politique, fichier métriques

  On précise que chaque test correspond au lancement de plusieurs algorithmes d'ordonnancement pour les mêmes fichiers d'entrée (mêmes processus, mêmes ressources de calcul, même énergie verte disponible).

## III. Organisation du travail

Le travail se déroule sur 5 semaines : deux semaines sur la partie Python, 2 semaines sur la partie Java et une semaine de finalisation et rendu de délivrables. Des oraux de présentation de groupe et individuels sont la 6ème semaine. Un contrôle sur table est la 7ème semaine (QCM et questions ouvertes).
Vous trouverez sur moodle le fichier travail-livrables qui décrit le déroulement et les étapes attendues.

Avant d'évaluer les compétences acquises, on vous demande de vous organiser de la manière suivante :

- pour python : trois sous-groupes
    - un sur les lectures, écritures de fichier
    - un groupe par politique d'ordonnancement (2 sous-groupes)
- pour java : 4 sous-groupes
    - un pour le paramétrage de la configuration
    - un pour les affichages généraux
    - un pour le diagramme de gantt par processus
    - un pour le diagramme de gantt par processeur

## IV. Fichiers de données en entrée

On décrit ci-dessous les informations à avoir dans les différents fichiers manipulés par l'application. Les formats sont imposés, les fichiers sont sur moodle. On raisonne temporellement en terme d'unités de temps. Le terme "date" indiqué dans

le sujet correspond à l'unité de temps à laquelle l'événement s'est produit (début ou fin de processus par exemple).

1. Données dans le fichier décrivant les ressources utilisables par l'ordonnanceur (fichier ressources.json). Il contient le nombre de processeurs total, la RAM totale disponible, la liste des identifiants de chaque processeur et l'énergie nécessaire pour faire tourner un processeur (on suppose que tous les processeurs consomment la même énergie).

2. Données dans le fichier décrivant les processus à exécuter (fichier processus.csv). Il contient pour chaque processus à ordonnancer : l'identifiant du processus, la date de soumission du processus, le temps d'exécution demandé, la RAM nécessaire pour l'exécution de ce processus, la priorité, la date butoire maximale (souvent appelée deadline) à laquelle le processus doit être fini.

3. Données dans le fichier décrivant l'énergie disponible par pas de temps (fichier energie.csv). Il contient pour chaque instant (pas de temps) l'énergie renouvelable disponible et utilisable par les processeurs.

4. Fichier de configuration (fichier config.txt). Il contient les fichiers d'entrée, de sortie, de métriques et le paramétrage du quantum pour les politiques en ayant besoin.

## V. Fichiers de données en sortie

Plusieurs fichiers stockent le résultat de l'ordonnancement :

1. fichier d'informations globales des processus : ce fichier doit contenir pour chaque processus : identifiant du processus, la date de soumission du processus, la date de début d'exécution (première date à laquelle le processus a obtenu le processeur), la date de fin d'exécution du processus, la RAM demandée et utilisée par le processus
2. fichier d'informations détaillées des processus sur les allocations processeurs : ce fichier doit contenir pour chaque processus : identifiant du processus, date de début d'exécution sur un processeur, date de fin d'exécution sur ce processeur, identifiant du processeur

3. fichier des métriques globales :
    a. métriques globales de l'ordonnancement : temps attente moyen, temps de réponse moyen, makespan
    b. métriques liées à l'énergie renouvelable disponible : énergie consommée totale (somme de l'énergie consommée par chaque processeur actif sur tous les pas de temps), énergie disponible totale sur la durée de simulation, taux d'utilisation (énergie consommée / énergie disponible).

## V. Politiques d'ordonnancement

Plusieurs politiques d'ordonnancement sont implémentables, nous en décrivons quelques unes ci-dessous.
Toutes les politiques sont basées sur la notion de file d'attente : file dans laquelle sont placés tous les processus prêts à s'exécuter à un instant donné.
Nous considérons qu'il y a une seule file pour gérer l'accès aux différents processeurs. On cherche dans la file quel processus peut s'exécuter sur chaque processeur que l'on considère l'un après l'autre.
Exemple : S'il y a 3 processeurs, les trois premiers processus de la file auront chacun un processeur différent. Le processus en 4ème position dans la file devra attendre qu'un processeur se libère.
Nous ferons l'hypothèse suivante : pour les politiques nécessitant un tri entre les processus arrivés par exemple à la même date, on considérera que la priorité sert à les classer dans la file d'attente. La priorité est une valeur numérique entière, plus la valeur est élevée et plus le processus est prioritaire.

1. Politique round-robin : cette politique considère la file d'attente dans l'ordre dans lequel ils y ont été insérés. Elle alloue un quantum de temps (durée constante dans la politique) au premier processus de la file puis lorsque la durée est atteinte, remet le processus en fin de file d'attente. Elle prend ensuite le processus suivant dans la file...

2. Politique FIFO : cette politique considère la file d'attente dans l'ordre dans lequel ils y ont été insérés. Si plusieurs processus ont la même date de soumission, c'est leur priorité qui impose un ordre d'ajout dans la file d'attente. Elle alloue le processeur au premier processus de la file jusqu'à ce que le processus ait terminé son exécution. Le processus "sort du système"

et n’est pas remis dans la file. Elle passe ensuite au processus suivant dans la file...

3. Politique par priorité: cette politique se base sur un tri de la file d’attente par priorité. Un processus de forte priorité pourra donc être inséré dans la file d’attente avant un processus déjà présent à condition d’être plus prioritaire.
   Elle alloue le processeur au premier processus de la file.
   Plusieurs variantes existent ensuite, vous devrez définir la variante que vous implémentez et vous pourrez en implémenter plusieurs pour comparer.
   Variante 1 : le processeur est alloué jusqu’à ce que le processus ait terminé son exécution. Le processus “sort du système” et n’est pas remis dans la file. Elle passe ensuite au processus suivant dans la file...
   Variante 2 : le processeur est alloué pendant un quantum de temps puis sa priorité est modifiée et le processus est réinséré dans la file à la place correspondant à sa nouvelle priorité. Elle passe ensuite au processus suivant dans la file...

4. Politique basée sur la date butoir demandée : EDF (Earliest Deadline First) : cette politique se base sur un tri de la file d’attente par date butoir (deadline). Plus la date butoir du processus est proche et plus le processus est en début de file d’attente.
   Elle alloue le processeur au premier processus de la file.
   Le processeur est alloué jusqu’à ce que le processus ait terminé son exécution. Le processus “sort du système” et n’est pas remis dans la file. Elle passe ensuite au processus suivant dans la file...

Pour chaque politique, on prend en compte deux contraintes simultanées :

- la contrainte de la RAM (partagée entre les processeurs) : on attribuera le processeur à un processus que si la RAM disponible est supérieure ou égale à la RAM demandée par le processus. Si le processus en début de file d’attente ne peut pas être exécuté faute de RAM, on considère le processus suivant dans la file jusqu’à en trouver un pouvant s’exécuter.
- la contrainte de l’énergie renouvelable disponible. À chaque pas de temps, une quantité d’énergie est disponible ; chaque CPU en fonctionnement consomme une énergie sur un pas de temps donnée dans le fichier ressources.json. Ainsi, à chaque instant un nombre limité de CPU sont alimentables. Parmi les processus

prêts, on en sélectionne au plus le nombre atteignant la consommation des CPU alimentables.